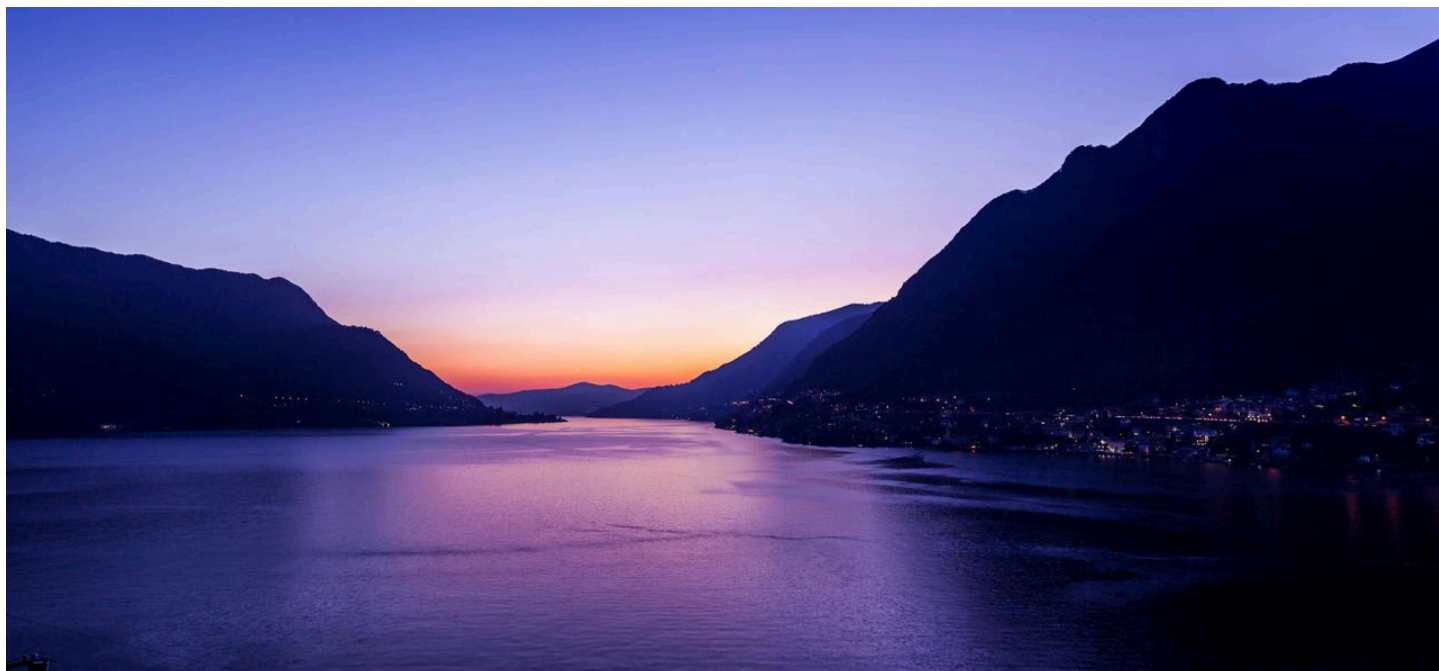




从零开始实现ddpm

最近期末月，同时也被ddpm的繁琐的数学推导搞得有点头大，理解了但是感觉写出来这篇文档还是很耗时的。。。最后磨蹭半天终于准备写一下。

数学原理：

ddpm总体分为两个过程，分别是前向传播和反向传播的过程。前向传播是一个加噪的过程，而反向传播是一个去噪的过程。

前向传播：

这里首先我们要知道一张图 x_0 ，是满足高斯采样的，，而后面在 t 的时间步里面生成的图片我们记为 x_t ，那么，我们加噪的过程化成公式就是：

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

这里 β_t 很小，基本上是从 10^{-4} 到 0.02 ，接下来我们引入 $\alpha_t = 1 - \beta_t$ ，所以形式上我们得到：

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon$$

这里epsilon是我们引入的随机噪声，满足 $N(0, I)$ 高斯分布，所以 x_t 依然满足高斯分布。

由于这显然很像一个马尔科夫链，所以我们可以得到 x_0 到 x_t 的公式，

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

其中 $\bar{\alpha}_t = \alpha_1 \alpha_2 \cdots \alpha_t$ ，浅显一点就是， $x_t = \text{保留比例} \times x_0 + \text{噪声比例} \times \epsilon$ ，当 t 很大的时候，第一项趋近于0，第二项趋近于1，可以认为是纯噪声。

反向传播：

反向传播就是和前向传播相反的过程，但是不同的是，这里需要模型进行预测（前向基本用于训练，反向用于采样），这里预测的只有一个噪声，原因会在后续的公式说明。

在前向里面对应的公式变成了

$$p_{\theta}(x_{t-1} | x_t)$$

DDPM：假设反向也是高斯分布，同时会固定方差，让网络预测均值。

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

所以神经网络学的是预测均值。

这里说了预测均值，为什么之前又说预测噪声呢，是因为：

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

预测噪声自然会有均值。

这样之后我们看一下采样的过程，主要是这个公式：

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z$$

这个 z 是一个随机在噪声，在 $t=1$ 的时候不加，加噪是为了保证多样性。

现在完整推导这个过程：

先准备两个已知分布：

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I)$$

$$q(x_{t-1} | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I)$$

然后用贝叶斯公式：

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0)q(x_{t-1} | x_0)}{q(x_t | x_0)}$$

即：

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1})q(x_{t-1} | x_0)}{q(x_t | x_0)}$$

其中分母和 x_{t-1} 无关(最后作为归一化常数)，而分子显然还是高斯分布。

然后我们处理分子，化简分子可得：

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t$$

但是这个时候我们 x_0 是不知道的，但是由于

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

故我们预测噪声，是为了利用这个公式（用预测噪声替代真实的）：

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

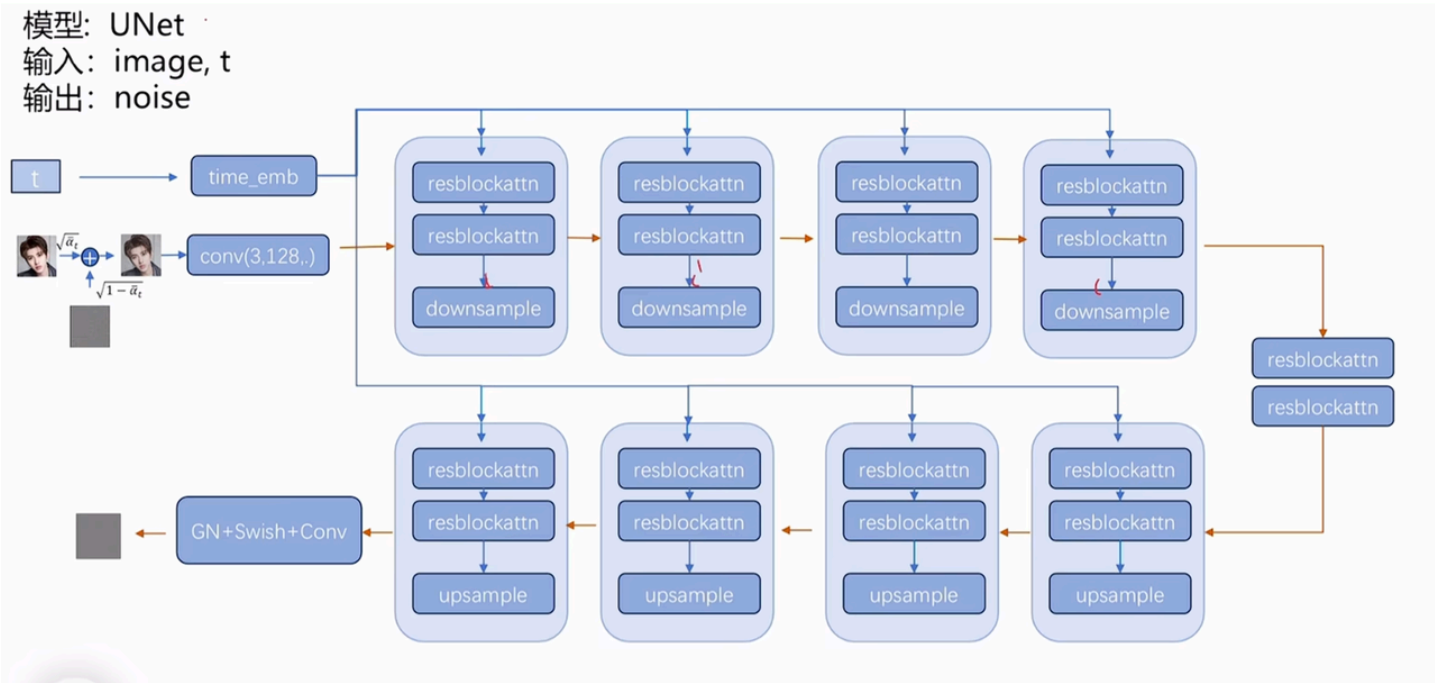
所以把这个x_0回代有

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

好的数学原理到这里就结束了，下面开始写代码了。

总体框架：

首先我们来看一下大体的架构，这里的ddpm是基于unet实现的，



代码实现：

1.首先是第一部分是time_embedding:

代码块

```

1  class TimeEmbedding(nn.Module):
2      def __init__(self,T,d_model,dim):
3          super().__init__()
4          emb = torch.arange(0, d_model, step = 2).float()/d_model *
math.log(10000)
5          emb = torch.exp(-emb)
6
7          pos = torch.arange(T).float().unsqueeze(1)
8          #这个的意思是加维度
9          emb = emb.unsqueeze(0)
10         emb = pos * emb #为什么要乘起来?
11         emb = torch.stack([torch.sin(emb),torch.cos(emb)],dim=-1)
12         emb = emb.view(T, d_model)#这向量是怎么个变化?
13         self.timeembedding = nn.Sequential(
14             nn.Embedding.from_pretrained(emb,freeze=False),#这个的意思是把上面算好
的表作为初始权重, freeze=False表示在训练过程中这个权重是可以更新的。
15             nn.Linear(d_model,dim),
16             nn.SiLU(),
17             nn.Linear(dim,dim)
18         )
19         self.initialize()
20
21     def initialize(self):
22         for _ in self.modules():
23             if isinstance(_,nn.Linear):
24                 init.xavier_uniform_(_.weight)
25                 init.zeros_(_.bias)
26     def forward(self,t):
27         return self.timeembedding(t)

```

这里顺便定义了激活函数和提取函数：

代码块

```

1  def extract(v, t, x_shape):
2      out = v.gather(dim =0, index=t)
3      return out.float().view(t.shape[0],*((1,)*(len(x_shape)-1)))
4      #这个return的向量形状不是很清楚,为什么要这么变形?
5      #因为这个函数的作用是从长度为 T 的一维系数表 v 中,按 batch 内每张图片的时间步 t 取
出对应系数。返回的向量形状是 [B, 1, 1, 1],这样可以和图片张量 [B, C, H, W] 自动广播相
乘。
6
7  class Swish(nn.Module):
8      def forward(self,x):
9          return x*torch.sigmoid(x)

```

2.第二部分是注意力机制，不过用的是卷积版，还有有意思的：

代码块

```
1  class AttnBlock(nn.Module):
2      def __init__(self, in_ch):
3          super().__init__()
4          self.groprnorm = group_norm(in_ch)
5
6          self.proj_q = nn.Conv2d(in_ch, in_ch, kernel_size=1, stride=1, padding=0)
7          self.proj_k = nn.Conv2d(in_ch, in_ch, kernel_size=1, stride=1, padding=0)
8          self.proj_v = nn.Conv2d(in_ch, in_ch, kernel_size=1, stride=1, padding=0)
9          self.proj = nn.Conv2d(in_ch, in_ch, kernel_size=1, stride=1, padding=0)
10         self.initialize()
11
12     def initialize(self):
13         for module in [self.proj_q, self.proj_k, self.proj_v, self.proj]:
14             init.xavier_normal(module.weight)
15             init.zeros_(module.bias)
16
17             init.xavier_normal(self.proj.weight, gain = 1e-5)
18
19     def forward(self, x):
20         B, C, H, W = x.shape
21         h = self.groprnorm(x)
22
23         q = self.proj_q(h)
24         k = self.proj_k(h)
25         v = self.proj_v(h)
26
27         q = q.permute(0, 2, 3, 1).view(B, H*W, C)
28         k = k.view(B, C, H*W)
29
30         w = torch.matmul(q, k) * (int(C) ** -0.5)
31
32         w = F.softmax(w, dim = -1) #dim=-1的含义?是指在最后一个维度上进行softmax操作，也就是对每个像素位置的注意力权重进行归一化，使得它们的和为1。
33
34         v = v.permute(0, 2, 3, 1).view(B, H*W, C)
35
36         score = torch.matmul(w, v)
37         score = score.view(B, H, W, C).permute(0, 3, 1, 2)
38         score = self.proj(score)
39
40         return score + x
41
```

3.第三部分是框架图里面的Resblock的结构：

主要作用就是把时间步和UNet里面上下采样的结果融合起来：

代码块

```
1  class Resblock(nn.Module):
2      def __init__(self,in_ch,out_ch,tdim,dropout,attn = False):
3          super().__init__()
4
5          self.block1 = nn.Sequential(
6              group_norm(in_ch),
7              Swish(),
8              nn.Conv2d(in_ch,out_ch,kernel_size=3,stroke=1,padding=1),#为什么要用
          3*3的卷积核?
9          )
10
11         self.tdim_proj = nn.Sequential(
12             Swish(),
13             nn.Linear(tdim,out_ch),
14         )
15
16         self.block2 = nn.Sequential(
17             group_norm(out_ch),
18             Swish(),
19             nn.Dropout(dropout),
20             nn.Conv2d(out_ch,out_ch,3,stroke=1,padding=1),
21         )
22
23
24         if in_ch != out_ch:
25             self.shortcut = nn.Linear(in_ch,out_ch)
26         else :
27             self.shortcut = nn.Identity()
28
29         self.attn = AttnBlock(out_ch) if attn else nn.Identity()
30         self.initialize()
31
32     def initialize(self):
33         for _ in self.modules():
34             if isinstance(_, (nn.Conv2d,nn.Linear)):
35                 init.xavier_normal_(_.weight)
36                 init.zeros_(_.bias)
37
38     def forward(self,x,temb):#为什么这个temb有两维呢? 因为这个temb是时间嵌入的结果,
通常是一个二维张量,形状为 [B, tdim], 其中 B 是批量大小, tdim 是时间嵌入的维度。这个时间
嵌入向量会被映射成与图像特征图相同的通道数,并通过广播机制加到特征图上,以便在每个时间步都
能提供时间信息。
```

```

39         h = self.block1(x)
40         h = h + self.tdim_proj(temb)[:,:,:None,None]
41         h = self.block2(h)
42         h = h + self.shortcut(h)
43
44         h = self.attn(h)
45
46         return h

```

4.第四部分就是经典UNet了：

这个之前实现过一次，不多赘述了，敲的过程遇到的问题写在注释里面了：

代码块

```

1  class DownSample(nn.Module):
2      def __init__(self,ch):
3          super().__init__()
4          self.main = nn.Conv2d(ch,ch,kernel_size= 3 ,stride= 2 ,padding=1)
5          init.xavier_normal(self.main.weight)
6          init.zeros_(self.main.bias)
7
8      def forward(self,x):
9          x = self.main(x)
10         return x
11
12  class UpSample (nn.Module):
13      def __init__(self,ch):
14          super().__init__()
15          self.main = nn.Conv2d(ch,ch,3,stride=1,padding=1)
16          init.xavier_normal(self.main)
17          init.zeros_(self.main.bias)
18
19      def forward(self,x):
20          x = F.interpolate(x,scale_factor=2,mode='nearest')
21          return self.main(x)
22
23
24  #UNet
25  class UNet(nn.Module):
26      def __init__(self,
27          T,
28          ch = 64,
29          ch_mult=(1,2,2),#这是干什么的？ 这是每个分辨率层级的通道倍率；(1, 2, 2) 表示
30          64 -> 128 -> 128。

```

```

30         attn= (1,), #这个是干什么的？为什么加逗号？这是一个元组，表示在哪些层级使用注意力
                        机制；这里默认第 1 层级，也就是 14x14 附近。加逗号是为了让 Python 识别这是一个单元素的元
                        组，而不是一个普通的括号表达式。
31         num_res_blocks=2,
32         dropout = 0.1,
33         in_ch=1,
34         out_ch=1,
35     ):
36
37         super().__init__()
38         self.T = T
39         self.ch=ch
40         self.tdim=ch*4 #为什么*4？这是时间嵌入的维度，通常设置为基础通道数的4倍，以提供
                        足够的表达能力。
41         self.num_res_blocks = num_res_blocks
42
43         self.time_embedding=TimeEmbedding(T,ch,self.tdim)
44         self.head = nn.Conv2d(in_ch,ch,3,stride=1,padding=1)
45     #Encoder部分
46         self.downblocks = nn.ModuleList()
47         chs = [ch]
48         now_ch = ch
49         for i,mult in enumerate(ch_mult):
50             out_channels = ch*mult
51             for _ in range(num_res_blocks):
52
53                 self.downblocks.append(ResBlock(now_ch,out_channels,self.tdim,dropout,attn=(i
54                 in attn)))
55                 now_ch = out_channels
56                 chs.append(now_ch)
57
58                 if i != len(ch_mult)-1:
59                     self.downblocks.append(DownSample(now_ch))
60                     chs.append(now_ch)
61     #Middle:
62         self.middleblocks = nn.Module([
63             ResBlock(now_ch,now_ch,self.tdim,dropout,attn=True),
64             ResBlock(now_ch,now_ch,self.tdim,dropout,attn=False),
65         ])
66
67     #Decoder:上采样过程
68         self.upblocks = nn.ModuleList()
69         for i,mult in reversed(list(enumerate(ch_mult))):
70             out_channels=ch*mult
71             for _ in range(num_res_blocks+1):

```

```

72         skip_ch = chs.pop() #这是干什么? chs.pop()是什么用法? 这是从列表
    chs 中弹出最后一个元素, 表示当前层级对应的 skip connection 的通道数。这个通道数会和当前
    特征图的通道数相加, 作为 ResBlock 的输入通道数。
73
    self.upblocks.append(ResBlock(now_ch+skip_ch,out_channels,self.tdim,dropout,att
n=(i in attn)))
74         now_ch = out_channels
75
76         if i != 0:
77             self.upblocks.append(UpSample(now_ch))
78
79         self.tail = nn.Sequential(
80             group_norm(now_ch),
81             Swish(),
82             nn.Conv2d(now_ch,out_ch,3,stride=1,padding=1),
83         )
84         self.initialize()
85
86         def initialize(self):
87             init.xavier_normal(self.head.weight)
88             init.zeros_(self.head.bias)
89             init.xavier_normal(self.tail[-1].weight,gain=1e-5)
90             init.zeros_(self.tail[-1].bias)
91
92         def forward(self,x,t):
93             temb = self.time_embedding(t)
94
95             h = self.head(x)
96             hs = [h]
97
98             for layer in self.downblocks:
99                 if isinstance (layer,ResBlock):
100                     h = layer(h,temb)
101                 else :
102                     h = layer(h)
103                 hs.append(h)
104
105             for layer in self.middleblocks:
106                 h = layer(h,temb) #为什么这个layer只要俩个参数? 因为 middleblocks 中
    的 ResBlock 只需要图像特征和时间嵌入作为输入, 不涉及下采样或上采样操作, 所以不需要额外的
    参数。
107
    #为什么可以把h整个传进去? 因为 ResBlock 的 forward 方法定义为 def
    forward(self, x, temb), 其中 x 是图像特征, temb 是时间嵌入。无论是在 encoder 还是
    middle 部分, 输入到 ResBlock 的都是当前的图像特征 h 和时间嵌入 temb, 所以直接把 h 传进
    去就可以了。
108
    #这个layer是什么东西? 其他的layer也是resblock吗? 这个layer是
    middleblocks 中的 ResBlock 实例, 其他的 layer 可能是 downblocks 中的 ResBlock 或

```

DownSample, upblocks 中的 ResBlock 或 UpSample。根据 isinstance 的判断, 代码会自动区分不同类型的 layer, 并传入相应的参数。

```
109
110
111         #Decoder :
112         for layer in self.upblocks:
113             if isinstance(layer, ResBlock):
114                 skip = hs.pop
115                 if h.shape[-2:] != skip.shape[-2:]: #h的形状是什么? skip的形状
                    是什么? h 的形状是当前的图像特征图, 通常是 [B, C, H, W]; skip 的形状是对应的 skip
                    connection 特征图, 通常也是 [B, C_skip, H_skip, W_skip]。如果它们的空间尺寸不一致, 就
                    需要通过插值对齐。
116                 h =
                    F.interpolate(h, size=skip.shape[-2:], mode='nearest')
117
118                 h = torch.cat([h, skip], dim=1)
119                 h = layer(h, temb)
120             else :
121                 h = layer(h)
122
123         return self.tail(h)
```

5.终于到DDPM的环节了, 这里先写训练器:

训练器主要是ddpm很像里面的前向的过程, 先用前向公式导出 x_t , 然后让网络预测的噪声接近我们加的噪声:

代码块

```
1  class GaussianDiffusionTrainer(nn.Module):
2      def __init__(self, model, bata_1 = 1e-4, beta_T = 0.02, T=1000):
3          super().__init__()
4          self.model = model
5          self.T = T
6          self.register_buffer('betas', torch.linspace(bata_1, beta_T, T).float()) #
            这个的意思是创建一个长度为 T 的一维张量, 线性地从 beta_1 增加到 beta_T, 并注册为模型的
            buffer, 这样它就会随着模型一起保存和加载, 但不会被优化器更新。
7          #为什么beta_T这么小? 因为 beta_T 是扩散过程最后一步加入的噪声强度, 设置得太大可
            能会导致生成的图像质量下降; 设置得太小可能会导致训练不稳定。通常在 0.01 到 0.02 之间是比
            较常见的选择。
8          alphas = 1.0 - self.betas
9          alphas_bar = torch.cumprod(alphas, dim=0) #这个函数的作用是计算 alphas 的累
            积乘积, 得到 alpha_bar_t = alpha_1 * alpha_2 * ... * alpha_t, 这个值在训练公式中用于
            计算 x_t 和噪声的权重。
10         self.register_buffer('sqrt_alphas_bar', torch.sqrt(alphas_bar))
```

```

11         self.register_buffer('sqrt_one_minus_alphas_bar', torch.sqrt(1.0-
    alphas_bar)) #这一步是？ 这是为了在训练公式中直接使用  $\sqrt{\alpha_{\bar{t}}}$  和  $\sqrt{1 - \alpha_{\bar{t}}}$ ，避免每次计算时都要进行平方根运算，提高效率。
12
13     def forward(self, x_0):
14         B = x_0.shape[0]
15
16         t = torch.randint(self.T, size = (B,), device=x_0.device) #这个函数的写
    法是这样的吗？ 每个参数的意义是什么？ 这是 PyTorch 中的一个函数，用于生成一个形状为 (B,)
    的整数张量，元素值在  $[0, T)$  的范围内，表示每张图片随机抽取的时间步。size 参数指定输出张量
    的形状，device 参数指定输出张量所在的设备 (CPU 或 GPU) 。
17
18         noise = torch.randn_like(x_0)
19         #这是什么函数？ 这是 PyTorch 中的一个函数，用于生成与 x_0 形状相同的张量，元
    素值服从标准正态分布（均值为 0，标准差为 1）。这个噪声张量是用来模拟扩散过程中的随机噪声
    的。
20
21         x_t = (
22             extract(self.sqrt_alphas_bar, t, x_0.shape)*x_0+
23             extract(self.sqrt_one_minus_alphas_bar, t, x_0.shape)*noise
24         )
25
26         pred_noise = self.model(x_t, t)
27         loss = F.mse_loss(pred_noise, noise, reduction="mean")
28         return loss

```

6.接下来是DDPM的采样器：

这里包含了模型预测噪声，然后实现了数学公式到代码的映射，最后得到我们预测的 x_0 ：

代码块

```

1 class GaussianDiffusionSampler(nn.Module):
2     def __init__(self, model, beta_1=1e-4, beta_T=0.02, T=1000):
3         super().__init__()
4         self.model = model
5         self.T = T
6
7         self.register_buffer('betas', torch.linspace(beta_1, beta_T,
    T).float())
8         alphas = 1.0 - self.betas
9         alphas_bar = torch.cumprod(alphas, dim=0)
10        alphas_bar_prev = F.pad(alphas_bar, [1, 0], value=1.0)[:T]
11        #这里面每个参数是什么意思？ 这是为了计算反向分布的方差时需要用到的  $\alpha_{\bar{t}-1}$ ，由于  $\alpha_{\bar{t}}$  的第 0 项对应  $t=0$  时的值为 1，所以通过在前面填充一个 1.0 来实现对

```

齐。pad 参数 [1, 0] 表示在第一个维度前面填充 1 个元素, 后面不填充; value=1.0 表示填充的值为 1.0; [:T] 是为了去掉多余的最后一项, 使得 alphas_bar_prev 的长度与 T 一致。

```
12
13
14     self.register_buffer('coeff1', torch.sqrt(1.0/alphas))
15     self.register_buffer('coeff2', self.coeff1*(1.0 -
alphas)/torch.sqrt(1.0-alphas_bar))
16     self.register_buffer('posterior_var', self.betas * (1.0 -
alphas_bar_prev) / (1.0 - alphas_bar))
17
18     def predict_xt_prev_mean_from_eps(self, x_t, t, eps):
19         return (
20             extract(self.coeff1, t, x_t.shape)*x_t-
21             extract(self.corff2, t, x_t.shape)*eps
22         )
23
24     def p_mean_varince(self, x_t, t):
25         var = torch.cat([self.posterior_var[1:2], self.betas[1:]]) #这一行什么意
意思? 这是为了处理 t=0 时 posterior_var 为 0 的情况, 避免数值不稳定。通过将
posterior_var 的第 1 项 (对应 t=1 的值) 替代第 0 项, 使得在 t=0 时也有一个合理的方差
值, 从而保证采样过程的稳定性。
26         var = extract(var, t, x_t.shape)
27
28         eps = self.model(x_t, t)
29         xt_prev_mean = self.predict_xt_prev_mean_from_eps(x_t, t, eps=eps)
30         return xt_prev_mean, var
31
32     @torch.no_grad()
33     def forward(self, x_T):
34         x_t = x_T
35         for time_step in reversed(range(self.T)):
36             t = x_t.new_full((x_t.shape[0],), time_step, dtype = torch.long)
37             mean, var = self.p_mean_varince(x_t=x_t, t=t)
38
39             if time_step > 0:
40                 noise = torch.randn_like(x_t)
41
42             else :
43                 noise = 0
44
45             x_t = mean + torch.sqrt(var)*noise
46
47         x_0 = x_t
48         return torch.clip(x_0, -1.0, 1.0) #clip在干什么? 这是为了确保生成的图像像素值
在 [-1, 1] 的范围内, 因为在训练阶段我们将输入图像归一化到了这个范围。clip 函数会将 x_0
中的值限制在指定的范围内, 超过 -1.0 的部分会被设置为 -1.0, 超过 1.0 的部分会被设置为
1.0, 从而保证输出的图像数据合法。
```

小结：

这次train和inference的过程依旧没写hh（），最近期末周+一堆ddl时间有点紧张，周末学完ddpm之后这周几乎没什么实质性推进，后面还是要合理规划一下时间。

从ddpm的架构来看，感觉挺老的，毕竟用的还是CNN+UNet哈哈，而且这种一步一步还原噪声的过程本身感觉就是一种比较低效的东西，虽然有batch，但是感觉天然不如transformer的训练和推理来的高效。